

Exercice 1 : P et NP

Que signifie qu'un problème p est dans P ? Un problème p est dans la classe P s'il peut être résolu par un algorithme déterministe en temps polynomial. Cela signifie qu'il existe un algorithme qui peut trouver la solution au problème en un temps proportionnel à une puissance polynomiale de la taille de l'entrée. Les problèmes dans P sont généralement considérés comme "facilement" solubles par un ordinateur.

Que signifie qu'un problème p est dans NP ?

Un problème p est dans la classe NP (Nondeterministic Polynomial time) s'il peut être vérifié par un algorithme déterministe en temps polynomial. Autrement dit, si on nous donne une solution candidate pour le problème, nous pouvons vérifier si cette solution est correcte en temps polynomial. Les problèmes de la classe NP peuvent potentiellement être résolus par un algorithme non déterministe en temps polynomial, ce qui signifie qu'un hypothétique ordinateur non déterministe pourrait deviner une solution et la vérifier en temps polynomial.

Exemples de problèmes

Problème dans P : Tri par insertion (Insertion Sort) : C'est un algorithme de tri simple qui fonctionne en construisant progressivement une liste triée en insérant les éléments un par un à la bonne position. Le tri par insertion a une complexité temporelle de $O(n^2)$ dans le pire des cas, mais reste polynomial et appartient donc à la classe P.

Problème dans NP : Problème du voyageur de commerce (Travelling Salesman Problem, TSP) : Le TSP consiste à trouver le plus court chemin possible qui visite un ensemble de villes exactement une fois et revient à la ville de départ. Vérifier une solution donnée (un circuit proposé) pour voir si c'est le plus court ou même juste pour calculer sa longueur et vérifier si elle est inférieure à une valeur donnée, peut se faire en temps polynomial. Cependant, trouver cette solution optimale est un problème qui n'a pas encore été prouvé comme étant résoluble en temps polynomial par un algorithme déterministe.

La question de savoir si $P=NP$ est l'un des plus grands mystères non résolus de l'informatique théorique.

Hypothèse $P \neq NP$: La majorité des chercheurs en informatique théorique pensent que P n'est pas égal à NP. Cela signifie qu'il existe des problèmes dans NP qui ne peuvent pas être résolus en temps polynomial par un algorithme déterministe, même s'ils peuvent être vérifiés en temps polynomial.

Hypothèse $P = NP$: Si jamais quelqu'un prouve que $P=NP$, cela signifierait que chaque problème pour lequel une solution peut être vérifiée en temps polynomial peut également être résolu en temps polynomial. Cela aurait des implications profondes et vastes pour de nombreux domaines, y compris la cryptographie, car beaucoup de systèmes de sécurité actuels reposent sur la difficulté de résoudre certains problèmes NP.

Exercice 2 : Machine de Turing

On démontre que $P_n = 2^n + 2^{n-1}$ est multiple de 3, pour $n > 0$. En factorisant P_n par 2^{n-1} , on trouve que $P_n = 2 * 2^{n-1} + 2^{n-1} = 2^{n-1} * (2 + 1) = 3 * 2^{n-1}$

Donc P_n est un multiple de 3.

Comme les différents P_i sont des multiples de 3, la somme des P_i est factorisable par 3, et est

donc multiple de 3 également. On en déduit que la somme $P_{i_1} + .. + P_{i_j} + P_{i_k} + ...P_{i_n}$ est multiple de 3, avec $abs(i_j - i_k) \geq 2$.

En binaire, une valeur 2^n est un nombre binaire avec un '1' positionné à la nieme position en partant de la droite. Par exemple $2^4 = 1000b$. Ainsi, les nombres dont on parle sont constitués de paires de '1' consécutifs. (par exemple 1100110011).

La machine de Turing M est définie par $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$, où :

- $Q = \{q_0, q_1, q_{accept}\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, B\}$ (où B est le symbole blanc)
- q_0 est l'état initial
- q_{accept} est l'état d'acceptation
- δ est la fonction de transition définie par :

δ	0	1	B
(q_0)	$q_0, 0, R$	$q_1, 1, R$	q_{accept}, B, R
(q_1)		$q_0, 1, R$	

Description de l'algorithme :

1. État initial q_0 :

- Si le symbole courant est '1', transition vers q_1 et déplacer la tête à droite. On a trouvé un '1', il faut en chercher un autre.
- Si le symbole courant est '0', rester en q_0 et déplacer la tête à droite.
- Si le symbole courant est B (fin de la bande), accepter.

2. État q_1 : Il nous faut absolument trouver un second 1, pour terminer la paire de '1'

- Si le symbole courant est '1', transition vers q_0 et déplacer la tête à droite.
- Si le symbole courant est '0' ou B, rejeter.

Exercice 3 : Matrices creuses

On appelle une matrice "creuse", une matrice pour laquelle la plupart (90%) des éléments a une valeur nulle. Ainsi, pour la représenter, on propose cette solution. Chaque élément de la matrice est explicitement indiqué. Les zéros occupent la majorité des cases, ce qui est typique pour une matrice creuse.

$$A = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

ou alors un tableau

```
tab = {
    (1, 10, 1),
    (2, 8, 2),
    (6, 7, 3),
    (8, 6, 4),
    (10, 1, 5)
}
```

Représentation par un tableau à une dimension avec des triplets, seuls les éléments non nuls sont stockés sous forme de triplets. Chaque triplet (l, c, v) représente la ligne, la colonne, et la valeur de l'élément non nul. Cette représentation est plus compacte et économise de l'espace, ce qui est avantageux pour les matrices creuses.

Pour une matrice de dimension $n \times n$, nous calculons la somme des différents éléments pour chaque représentation : tableau à deux dimensions et tableau à une dimension avec des triplets (l, c, v) .

```
def sum_matrix_2d(matrix):
    n = len(matrix)
    total_sum = 0
    for i in range(n):
        for j in range(n):
            total_sum += matrix[i][j]
    return total_sum
```

La complexité spatiale est $O(n^2)$. La matrice est stockée explicitement, nécessitant n^2 entiers, car chaque élément de la matrice est représenté, même s'il est nul.

La complexité temporelle est $O(n^2)$. Nous effectuons une somme pour chaque élément de la matrice. Le nombre total d'opérations est donc $n \times n = n^2$.

```
def sum_matrix_triplets(triplets):
    total_sum = 0
    for triplet in triplets:
        total_sum += triplet[2] # triplet[2] est la valeur v
    return total_sum
```

La complexité spatiale est $O(k)$, où k est le nombre d'éléments non nuls dans la matrice. Chaque élément non nul est stocké sous forme de triplet (l, c, v) . Le nombre d'entiers stockés est $3k$. Puisque k est bien inférieur à n^2 pour une matrice creuse, cette représentation est beaucoup plus économe en mémoire.

La complexité temporelle est $O(k)$. Nous effectuons une somme pour chaque élément non nul. Le nombre total d'opérations est donc proportionnel au nombre d'éléments non nuls k , ce qui est beaucoup plus efficace que n^2 .

- **Tableau à deux dimensions :**
 - Complexité spatiale : $O(n^2)$
 - Complexité temporelle : $O(n^2)$
- **Tableau à une dimension avec des triplets :**
 - Complexité spatiale : $O(k)$
 - Complexité temporelle : $O(k)$

Pour comparer les deux algorithmes, nous devons examiner leurs complexités spatiales et temporelles. Une comparaison directe montre que la méthode utilisant des triplets est plus efficace pour une matrice suffisamment creuse.

Pour déterminer la valeur spécifique du nombre d'éléments non nuls à partir de laquelle une méthode l'emporte sur l'autre, nous comparons k avec n^2 . Si k est significativement plus petit que n^2 , la méthode des triplets est préférable.

Preuve

Soit k le nombre d'éléments non nuls de la matrice :

— **Complexité Spatiale :**

- La méthode par tableau à deux dimensions stocke chaque élément, donc utilise n^2 entiers.
- La méthode par triplets utilise $3k$ entiers (chaque triplet représente un élément non nul). La méthode des triplets est plus efficace en espace mémoire si $3k < n^2$. Simplifions pour trouver le seuil :

$$k < \frac{n^2}{3}$$

— **Complexité Temporelle :**

- La méthode par tableau à deux dimensions effectue n^2 opérations.
 - La méthode par triplets effectue k opérations.
- La méthode des triplets est plus rapide si $k < n^2$.

Pour une matrice de dimension $n \times n$, la méthode des triplets est préférable si le nombre d'éléments non nuls k est inférieur à $\frac{n^2}{3}$. Au-delà de ce seuil, la méthode par tableau à deux dimensions devient plus compétitive en termes de complexité spatiale. En termes de complexité temporelle, la méthode des triplets reste préférable car $k < n^2$.

Exercice 4 : Complexité

Question 1 :

Proposez une formulation verbale et mathématique de la complexité de chacun de ces programmes P en fonction de n :

- **Programme en général inutilisable, sauf cas très spécifique où n est vraiment petit.** Ce programme a une complexité exponentielle, $O(2^n)$. Il devient rapidement inutilisable à mesure que n augmente, sauf pour des valeurs très petites de n . Exemple : résolution par force brute du problème du voyageur de commerce (TSP).
- **Programme qui devient 2 fois plus lent quand n est doublé.** Ce programme a une complexité linéaire, $O(n)$. Son temps d'exécution double chaque fois que n double. Exemple, un parcours simple d'un tableau de taille n .
- **Programme qui devient seulement légèrement plus lent quand n croît.** Ce programme a une complexité logarithmique $O(\log n)$. Son temps d'exécution augmente très lentement à mesure que n augmente. Exemple, recherche binaire dans un tableau trié.
- **Programme dont la complexité est typique du tri par insertion.** Ce programme a une complexité quadratique : $O(n^2)$. Son temps d'exécution augmente proportionnellement au carré de la taille de l'entrée.
- **Programme utilisable, qu'avec de petites valeurs.** : Ce programme a une complexité cubique, ou plus, : $O(n^3)$. Il devient rapidement impraticable pour des valeurs modérées de n .

- **Programme dont la complexité est typique pour les algorithmes de type « diviser pour régner ».** Chaque fois que n est doublé, la complexité est un peu plus que doublée. Ce programme a une complexité de type $O(n \log n)$. Les algorithmes de ce type divisent le problème en sous-problèmes de taille plus petite et combinent les résultats. Leur temps d'exécution est plus que linéaire mais moins que quadratique.

Question 2 :

Analysons ce qui fait cette fonction en utilisant la récursion.

Pour $n = 0$:

$$\text{mystere}(0) = 0$$

Pour $n > 0$:

$$\text{mystere}(n) = \text{mystere}(n-1) + (2n) - 1$$

Calculons les premiers termes pour observer un schéma :

$$\text{mystere}(1) = 1$$

$$\text{mystere}(2) = 4$$

$$\text{mystere}(3) = 9$$

$$\text{mystere}(4) = 16$$

En raisonnant par récurrence ; supposons que $\text{mystere}(n) = n^2$, dans ce cas
 $\text{mystere}(n+1) = \text{mystere}(n) + 2(n+1) - 1 = n^2 + 2 * n + 1 = (n+1)^2$

Donc,

$$\text{mystere}(n) = n^2$$

Chaque appel récursif ajoute une opération additionnelle, donc la complexité temporelle en termes d'additions est proportionnelle à n . Ainsi, la complexité de `mystere` en nombre d'additions est :

$$O(n)$$

La fonction `mystere2` est définie comme suit :

```
int mystere2(int n){  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
    return mystere(n) + mystere2(n/2);  
}
```

La complexité de `mystere2` peut être définie par

$$C_{\text{mystere2}}(n) = C_{\text{mystere}}(n) + C_{\text{mystere2}}\left(\frac{n}{2}\right)$$

Sachant que $C_{\text{mystere}}(n) = n$, on a :

$$C_{\text{mystere2}}(n) = n + C_{\text{mystere2}}\left(\frac{n}{2}\right) = n + \left(\frac{n}{2} + C_{\text{mystere2}}\left(\frac{n}{4}\right)\right)$$

Quand n est très grand,

$$C_2(n) = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \cdots + C_2(1)$$

$$C_2(n) = n \left(1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{n} \right)$$

La somme de cette série est :

$$C_2(n) = n \left(2 - \frac{1}{n} \right)$$

Pour les grandes valeurs de n , $\frac{1}{n}$ devient insignifiant, donc :

$$C_2(n) \approx 2n$$

Ainsi, la complexité temporelle de `mystere2` est $O(n)$, mais en pratique, elle est proche de $2n$.

Question 3 :

Pour calculer x^{13} en utilisant le moins de multiplications possible, nous pouvons utiliser la méthode d'exponentiation rapide, également connue sous le nom de "exponentiation par squaring". Cette méthode est particulièrement efficace pour les puissances de nombres entiers.

Voici un algorithme optimal pour calculer x^{13} avec un minimum de multiplications :

1. $x^2 = x \times x$ (1 multiplication)
2. $x^4 = x^2 \times x^2$ (1 multiplication)
3. $x^8 = x^4 \times x^4$ (1 multiplication)
4. $x^{12} = x^8 \times x^4$ (1 multiplication)
5. $x^{13} = x^{12} \times x$ (1 multiplication)

Ainsi, nous avons calculé x^{13} en utilisant seulement 5 multiplications.

Voici l'algorithme en pseudocode pour calculer x^{13} :

```
int power_13(int x) {
    int a = x * x;    // x^2
    int b = a * a;    // x^4
    int c = b * b;    // x^8
    int d = c * b;    // x^12
    int e = d * x;    // x^13
    return e;
}
```

Voici l'algorithme en pseudocode pour calculer x^n :

```
int power(int x, int n) {
    if (n == 0) return 1;
    if (n == 1) return x;
    if (n % 2 == 0) {
        int temp = power(x, n / 2);
        return temp * temp;
    } else {
        int temp = power(x, (n - 1) / 2);
        return x * temp * temp;
    }
}
```

Partons du principe que $x=13$, $n=2$, alors

$power(2, 13)$	$= 2 * \alpha * \alpha$	$\alpha = power(2, 6)$	<i>Multiplications</i>
$power(2, 6)$	$= \beta * \beta$	$\beta = power(2, 3)$	2
$power(2, 3)$	$= 2 * \eta * \eta$	$\eta = power(2, 1)$	1
$power(2, 1)$	$= 2$	$\eta = 2$	2
$power(2, 3)$	$= 2 * 2 * 2 = 8$	$\beta = 8$	
$power(2, 6)$	$= 8 * 8 = 64$	$\alpha = 64$	
$power(2, 13)$	$= 2 * 64 * 64 = 8192$		

Question 4 :

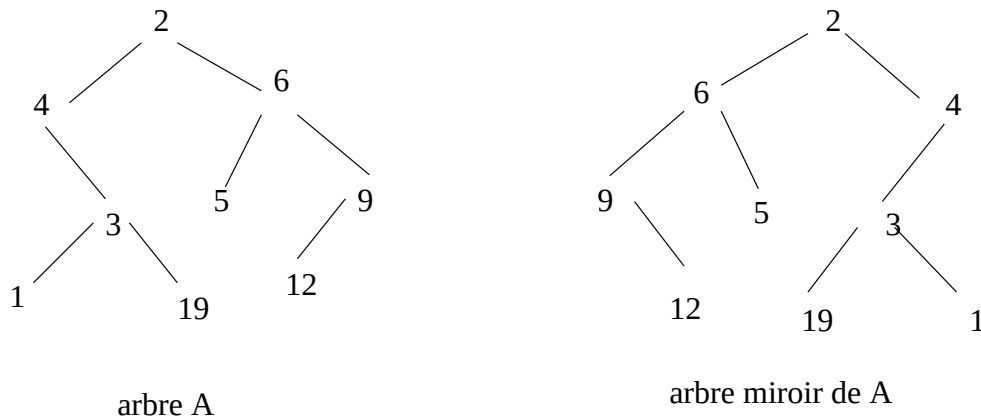


FIGURE 1 – Le miroir d'un arbre

Voici une fonction Python pour transformer un arbre binaire en son miroir :

```
class TreeNode:
    def __init__(self, value):
        self.val = value
        self.left = None
        self.right = None

def mirror_tree(root):
    if root is None:
        return
    root.left, root.right = root.right, root.left
    mirror_tree(root.left)
    mirror_tree(root.right)
```

La complexité de cette fonction en nombre d'appels récurifs dépend de la structure de l'arbre d'entrée.

- Chaque nœud a deux enfants. Dans ce cas, chaque nœud est visité une seule fois, et donc la complexité est $O(n)$, où n est le nombre de nœuds dans l'arbre.